

支持向量机

Support Vector Machine

明玉瑞 Yurui Ming

yrming@gmail.com

声明

Disclaimer

- ▶ 本讲义在准备过程中由于时间所限，所用材料来源并未规范标示引用来源。所引材料仅用于教学所用，作者无意侵犯原著者之知识产权，所引材料之知识产权均归原著者所有；若原著者介意之，请联系作者更正及删除。

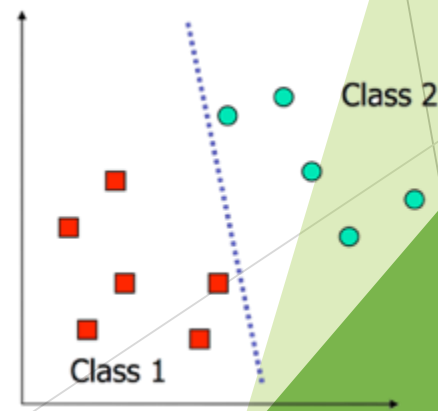
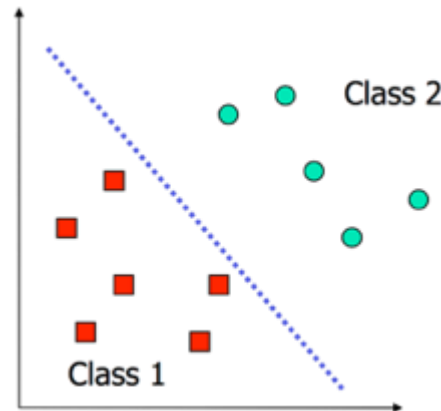
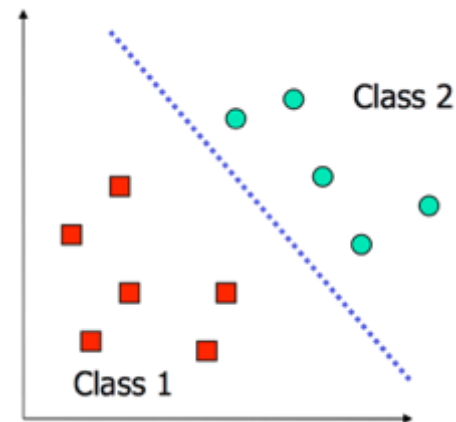
These slides have been developed free-of-charge for educational purposes only and without any commercial interest. However, time limit during the preparation incurs the situation that not all the sources of the materials (texts or images) are properly referenced or clearly manifested. But all contents are solely for teaching and the author is with no intention to infringe the copyright bestowed on the originators and all credits go to corresponding IP holders. Please address the author have you had any concern so he can act on a remedy including deletion per request.

决策边界

Decision Boundary

- 在讲支持向量机之前，我们先回顾一下之前学的知识。在逻辑回归中，我们会学习一个决策边界，用于在特征空间中划分训练样本的不同类别。当数据能够被线性边界完美分隔时，我们称该数据是线性可分的。在这种情况下，会有多条决策边界都能拟合数据。我们该如何选择最优的那一条？

Before we introduce the techniques about support vector machine, we first recap some knowledge we learned before. In logistic regression, we learn a decision boundary that separates the training classes in the feature space. When the data can be perfectly separated by a linear boundary, we call the data linearly separable. In this case, multiple decision boundaries can fit the data. How do we choose the best?

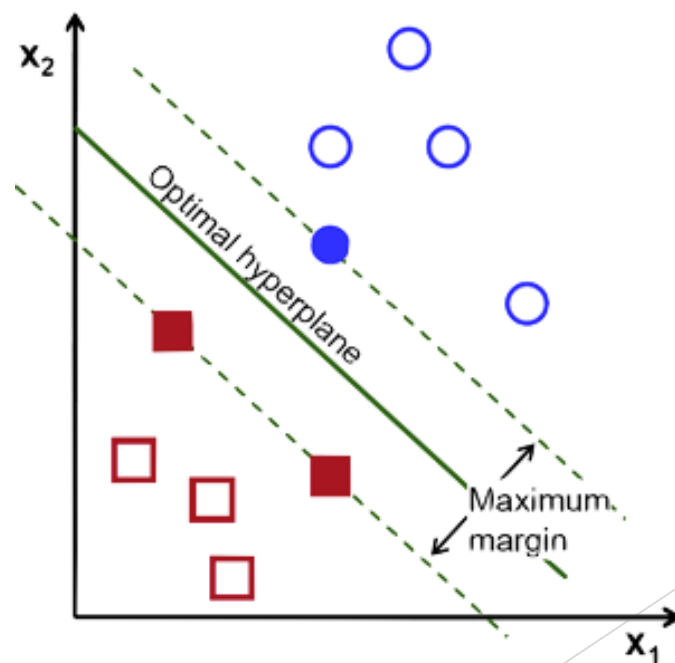


决策边界

Decision Boundary

- 那么，我们如何选择决策边界呢？一个直观的想法是，希望决策边界不“偏向”任何一个类别（尤其当各类别的样本数量大致相等时）。从几何角度来看，这意味着选择一条能最大化边界与两个类别之间距离（间隔）的决策边界。

Then how we choose the most appropriate decision boundary? Intuitively, we may prefer a decision boundary that does not “favour” any class (esp. when the classes are roughly equally populous). Geometrically, this means choosing a boundary that maximizes the distance or margin between the boundary and both classes.

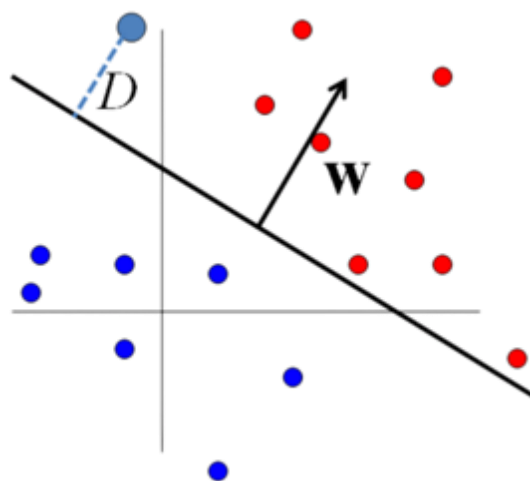


决策边界

Decision Boundary

- 在将上述表述数学化之前，我们先回顾一下决策边界的几何意义。决策边界由关于预测变量的某个方程所定义的，特别是线性决策边界，其为超平面的形式： $w^T x + b = 0$ ，其中 w 代表法向量，它垂直于超平面并指向外侧。

Before turning the above description into mathematical expression, we first recall the geometry of decision boundary. Recall that the decision boundary is defined by some equation in terms of the predictors. Specifically, a linear boundary is an equation of a hyperplane defined by $w^T x + b = 0$, and the non-constant coefficients, w , represent a normal vector, pointing orthogonally away from the plane.



决策边界

Decision Boundary

- 进一步的，我们从数学中知道，对于给定的 w 与 b ，对于空间中任意一点 $x \in \mathbb{R}^n$ ，其到超平面的有符号距离为 $D(x) = \frac{w^T x + b}{\|w\|}$ 。基于此，现在我们可以将目标——寻找一条到两个类别的距离都最大的决策边界——表述为如下一个优化问题：

In addition, from geometry, we can compute the distance between any point $x \in \mathbb{R}^n$ to the decision boundary with regard to the given w and b , which is $D(x) = \frac{w^T x + b}{\|w\|}$. And now we can formulate our goal - find a decision boundary that maximizes the distance to both classes - as the following optimization problem:

$$\begin{cases} \max_{w,b} M \\ \text{s. t. } |D(x_i)| = \frac{y_i(w^T x_i + b)}{\|w\|} \geq M \end{cases}$$

其中 M 是表示“间隔”宽度的实数，且 $y_i = \pm 1$ 。不等式 $|D(x_i)| \geq M$ 被称为约束条件。

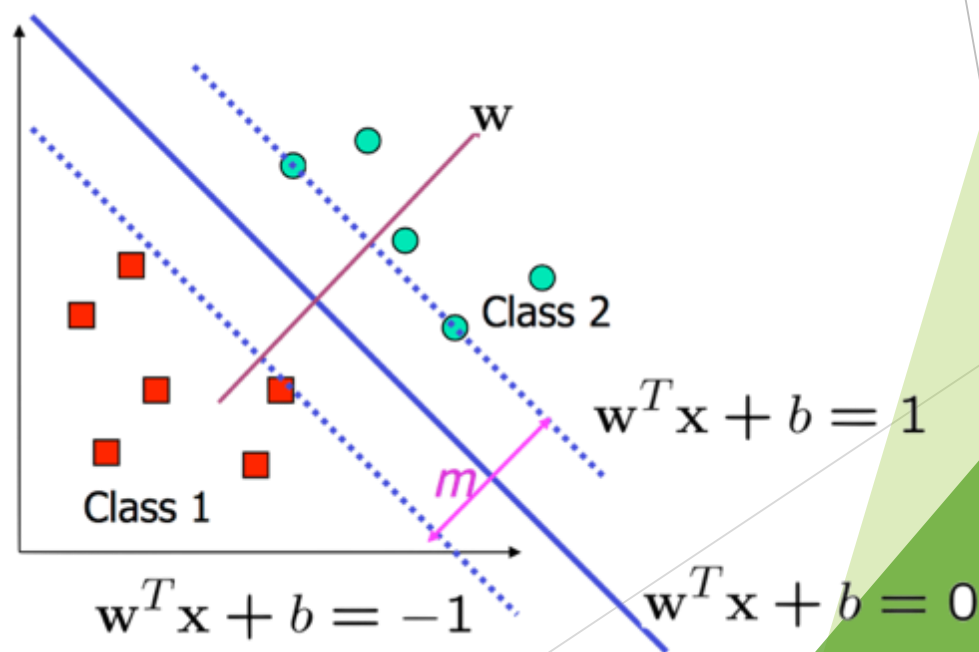
where M is a real number representing the width of the ‘margin’ and $y_i = \pm 1$. The inequalities $|D(x_i)| \geq M$ are called constraints.

支持向量机

Support Vector Machine

- 从几何上，我们不难直观地看出，最大化所有点到决策边界的距离，与最大化到最近点的距离是完全等价的。距离决策边界最近的这些点，被称为支持向量。对于任意超平面，我们总可以对其方程进行缩放： $w^T x + b = 0$ 使得支持向量落在平面 $w^T x + b = \pm 1$ 上。当然，具体取+1还是-1，取决于它们所属的类别。

From geometry, it is manifest that maximizing the distance of all points to the decision boundary, is exactly the same as maximizing the distance to the closest points. And the points closest to the decision boundary are called support vectors. For any plane, we can always scale the equation: $w^T x + b = 0$, so that the support vectors lie on the planes: $w^T x + b = \pm 1$, depending on their classes.



支持向量机

Support Vector Machine

- ▶ 显然，根据公式 $D(x) = w^T x + b / \|w\|$ ，落在直线 $w^T x + b = \pm 1$ （往往为超平面）上的点，其到决策边界的带符号距离为 $\pm 1 / \|w\|$ ，因此，我们可以将决策边界的间隔定义为它到支持向量的距离，即 $M = 2 / \|w\|$ 。因此，我们可以将前面的优化问题重新表述为：

Obviously, from the formula $D(x) = w^T x + b / \|w\|$, we know that for points on planes $w^T x + b = \pm 1$, their distance to the decision boundary is $\pm 1 / \|w\|$. So we can define the margin of a decision boundary as the distance to its support vectors, $M = 2 / \|w\|$. Therefor, we can reformulate our optimization problem in the following:

$$\begin{cases} \max_{w,b} \frac{2}{\|w\|} \\ \text{s.t. } y_i(w^T x_i + b) \geq 1, i = 1, 2, \dots, N \end{cases}$$

思考其与VC边界的联结
Consider its connection with VC bound.

在保持零误分类的前提下，最大化间隔 M 。有时，我们也写成极小化的形式：

Namely, to maximize the margin, M , while maintaining zero misclassifications. Sometimes we also present it as a minimization problem:

$$\begin{cases} \min_{w,b} \|w\|^2 \\ \text{s.t. } y_i(w^T x_i + b) \geq 1, i = 1, 2, \dots, N \end{cases}$$

硬间隔支持向量分类

Hard-margin Support Vector Classification

- ▶ 通过求解该问题得到的分类器，称为硬间隔支持向量分类。作为凸优化问题，支持向量分类已被广泛研究，可通过多种算法求解：

- ▶ （随机型）libLinear
收敛速度快，计算代价适中
- ▶ （贪心型）libSVM
收敛速度快，计算代价适中
- ▶ （随机型）随机梯度下降
收敛速度慢，单次迭代计算代价低
- ▶ （贪心型）拟牛顿法
收敛速度极快，计算代价高

- ▶ The classifier learned by solving this problem is called hard margin support vector classification. As a convex optimization problem SVC has been extensively studied and can be solved by a variety of algorithms:

- ▶ (Stochastic) libLinear
Fast convergence, moderate computational cost
- ▶ (Greedy) libSVM
Fast convergence, moderate computational cost
- ▶ (Stochastic) Stochastic Gradient Descent
Slow convergence, low computational cost per iteration
- ▶ (Greedy) Quasi-Newton Method
Very fast convergence, high computational cost

优化

Optimization

- ▶ 下面，我们简要说明一下求解SVC问题是可行的。我们先回顾一下关于优化的知识。在高中时，我们已经学过，关于二次曲线 $f(x) = ax^2 + bx + c$, $a \neq 0$, $x^* = -\frac{b}{2a}$

- ▶ 情形 1 : $f''(x^*) = 2a > 0 \Rightarrow x^* \in \arg \min_{x \in \mathbb{R}} f(x)$

- ▶ 情形 1 : $f''(x^*) = 2a > 0 \Rightarrow x^* \in \arg \min_{x \in \mathbb{R}} f(x)$

对于极小化问题（情形 I）

- ▶ $f'(x^*) = 0$ 称为一阶最优性条件
- ▶ $f''(x^*) > 0$ 称为二阶最优性条件

- ▶ Now we brief the feasibility of solving the SVC problem. We need to recap optimization first. In high school, we already know $f(x) = ax^2 + bx + c$, $a \neq 0$, $x^* = -\frac{b}{2a}$

- ▶ Case 1: $f''(x^*) = 2a > 0 \Rightarrow x^* \in \arg \min_{x \in \mathbb{R}} f(x)$

- ▶ Case 2: $f''(x^*) = 2a > 0 \Rightarrow x^* \in \arg \min_{x \in \mathbb{R}} f(x)$

For minimization problem (Case I),

- ▶ $f'(x^*) = 0$ is called the first order optimality condition.
- ▶ $f''(x^*) > 0$ is the second order optimality condition.

优化

Optimization

- 实际上，我们可以将上述结论进行拓展。令 $f: \mathbb{R}^n \rightarrow \mathbb{R}$, $f(x) = \frac{1}{2}x^T Hx + p^T x$, 这里 $H \in \mathbb{R}^{n \times n}$ 为对称矩阵, $p \in \mathbb{R}^n$ 。进一步地, $\nabla f = Hx + p$, $\Delta f = H$, 若 H 为正定矩阵, 则 $x^* = -H^{-1}p$ 为最小值; 若 H 为负定矩阵, 则 $x^* = -H^{-1}p$ 为最大值。

In fact, we can extend the above conclusion further. Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$, $f(x) = \frac{1}{2}x^T Hx + p^T x$, where $H \in \mathbb{R}^{n \times n}$ is a symmetric matrix and $p \in \mathbb{R}^n$. We have $\nabla f = Hx + p$, $\Delta f = H$ (Hessian), Note: If H is positive definite, then $x^* = -H^{-1}p$ is the unique solution of $\min f(x)$.

- 例 (Example) :

$$f(x) = x_1^2 + x_2^2 - 2x_1 + 4x_2 = \frac{1}{2} \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} -2 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$
$$\nabla f = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} -2 \\ 4 \end{bmatrix}, \Delta f = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$
$$x^* = -H^{-1}p = -\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}^{-1} \begin{bmatrix} -2 \\ 4 \end{bmatrix} = -\begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix} \begin{bmatrix} -2 \\ 4 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

优化

Optimization

- 上面我们考虑的是无约束的优化问题，但在实际中，我们面对的都是有约束的优化问题，即给定定义在 $\Omega \subset \mathbb{R}^n$ 上目标函数 $f(x)$ ，与约束函数 g_i 与 h_j ，这里 $i = 1, 2, \dots, k$ ； $j = 1, 2, \dots, m$ 。我们的目标是求解如下的优化问题：

The optimization problem we considered above is un-constraining. However, in practice we always constrained ones. Generally, give the objective function $f(x)$ which is defined on the domain $\Omega \subset \mathbb{R}^n$. Together with constraints g_i and h_j , here $i = 1, 2, \dots, k$, $j = 1, 2, \dots, m$. Our task is to solve the optimization below:

$$\begin{aligned} & \min_{x \in \mathbb{R}^n} f(x) \\ \text{s. t. } & g_i(x) \leq 0, \forall i \\ & h_j(x) = 0, \forall j \end{aligned}$$

$$\begin{aligned} \min f(x) &= 2x_1^2 + x_2^2 + 3x_3^2 \\ \text{s. t. } & 2x_1 - 3x_2 + 4x_3 = 49 \end{aligned}$$

$$L(x, \beta) = f(x) + \beta(2x_1 - 3x_2 + 4x_3 - 49), \beta \in \mathbb{R}$$

$$\frac{\partial}{\partial x_1} L(x, \beta) = 0 \Rightarrow 4x_1 + 2\beta = 0$$

$$\frac{\partial}{\partial x_2} L(x, \beta) = 0 \Rightarrow 2x_2 - 3\beta = 0$$

$$\frac{\partial}{\partial x_3} L(x, \beta) = 0 \Rightarrow 6x_3 + 4\beta = 0$$

$$\begin{aligned} 2x_1 - 3x_2 + 4x_3 - 49 &= 0 \Rightarrow -\beta - \frac{9}{2}\beta - \frac{8}{3}\beta \\ &= 49 \Rightarrow \beta = -6 \end{aligned}$$

$$\Rightarrow \begin{cases} x_1 = 3 \\ x_2 = -9 \\ x_3 = 4 \end{cases}$$

拉格朗日方法

Lagrange Method

- 上面的方法称为拉格朗日方法，其更为一般结论如下：

令 $\mathcal{L}(x, \alpha) = f(x) + \alpha^T g(x)$ ，这 $\alpha \geq 0$ 。如果 $f(x)$ 和 $g(x)$ 是凸函数，那么对于给定的 $\alpha \geq 0$ ，如果 $\bar{x} \in \arg \min\{\mathcal{L}(x, \alpha) | x \in \mathbb{R}^n\}$ ，我们有：

$$\left. \frac{\partial \mathcal{L}(x, \alpha)}{\partial x} \right|_{x=\bar{x}} = \nabla f(\bar{x}) + \alpha^T \nabla g(\bar{x}) = 0$$

这个结论中要求 $\mathcal{L}(x, \alpha)$ 为凸函数，因此只是求解的必要条件。但在实际求解中，如果样本维度较高，数目较大，求解过程会比较漫长。不过可以利用拉格朗日方法写出其对偶形式，降低求解的难度。

- The method above is called Lagrange method, a more general definition is in the below:

Let $\mathcal{L}(x, \alpha) = f(x) + \alpha^T g(x)$, here $\alpha \geq 0$. If $f(x)$ and $g(x)$ are convex, then for a fixed $\alpha \geq 0$, if $\bar{x} \in \arg \min\{\mathcal{L}(x, \alpha) | x \in \mathbb{R}^n\}$, we have:

$$\left. \frac{\partial \mathcal{L}(x, \alpha)}{\partial x} \right|_{x=\bar{x}} = \nabla f(\bar{x}) + \alpha^T \nabla g(\bar{x}) = 0$$

Above result is a sufficient condition since we require $\mathcal{L}(x, \alpha)$ is convex. However, in practice, solving the optimization problem according to Lagrange condition is intractable. But we can use the Lagrange method to deduce the dual form, which is easier than the direct form in general.

梯度下降

Gradient Descent

- 上面已经说过，按照解析计算方式求解，计算量太大，费时耗力。另外一种方式是采用梯度下降，但是由于是有约束的，因此，我们要建立相关理论。首先介绍可行域的概念：满足优化问题中约束条件的 x 组成的区域 $\mathcal{F}$ ，称为可行域，即：

As mentioned above, solving the problem using analytical methods is computation intensive and can be impractical in some situations. An alternative is to adopt gradient descent method. However, since the problem is constrained, we need to establish the relevant theory. First, we introduce the concept of the feasible region: the set $\mathcal{F}$ composed of all x that satisfy the constraints in the optimization problem is called the feasible region, namely:

$$\mathcal{F} = \{x \in \Omega | g(x) \leq 0, h(x) = 0\}$$

- 令 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 为连续可微的凸函数， $\mathcal{F} \subset \mathbb{R}^n$ 为可行域，则我们有下面结论成立：

Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be a convex and continuously differentiable function, $\mathcal{F} \subset \mathbb{R}^n$ be the feasible region, then we have the following conclusion:

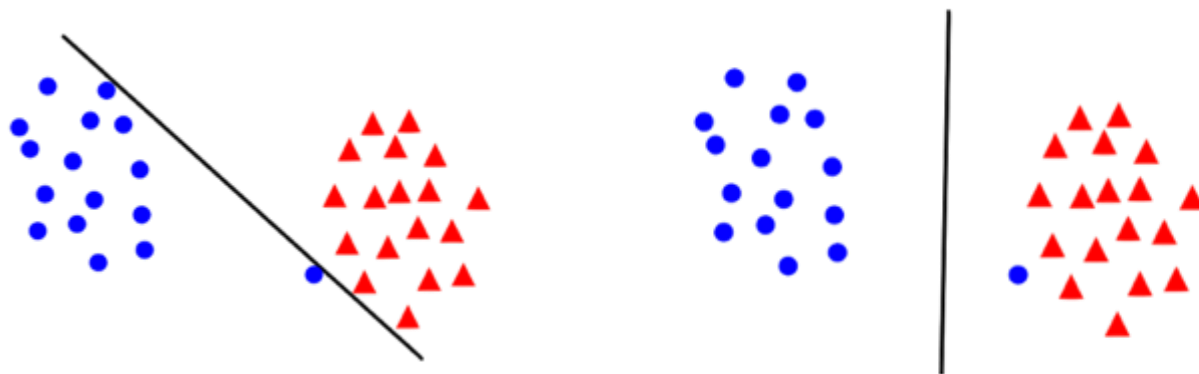
$$x^* \in \arg \min_{x \in \mathcal{F}} f(x) \Leftrightarrow \nabla f(x^*)(x - x^*) \geq 0, \forall x \in \mathcal{F}$$

软间隔

Soft Margin

- 如果数据的类别是线性可分的且数据不含噪声，那么最大化间隔就是一个很好的选择。但如果数据存在噪声，我们就可能为了用极窄的间隔最小化分类误差，而牺牲模型的泛化能力。因此，对于任意决策边界，都存在最大化间隔与最小化误差之间的权衡。

Maximizing the margin is a good idea as long as the underlying classes are linear separable and that the data is noise free. If data is noisy, we might be sacrificing generalizability in order to minimize classification error with a very narrow margin. Therefore, With every decision boundary, there is a trade-off between maximizing margin and minimizing the error.



软间隔

Soft Margin

- 由于我们希望在最大化间隔与最小化误差之间取得平衡，因此需要采用一个能同时兼顾两者的如下所示的目标函数，其中 λ 为强度参数：

Since we want to balance maximizing the margin and minimizing the error, we want to use an objective function that takes both into account as in the following, where λ is an intensity parameter:

$$\begin{cases} \min_{w,b} \|w\|^2 + \lambda \cdot \text{Error}(w, b) \\ \text{s.t. } y_i(w^T x_i + b) \geq 1, i = 1, 2, \dots, N \end{cases}$$

- 那么，对于给定的决策边界，我们应当如何计算误差？我们希望将误差表示为样本到决策边界距离的函数。

So just how should we compute the error for a given decision boundary? We want to express the error as a function of distance to the decision boundary.

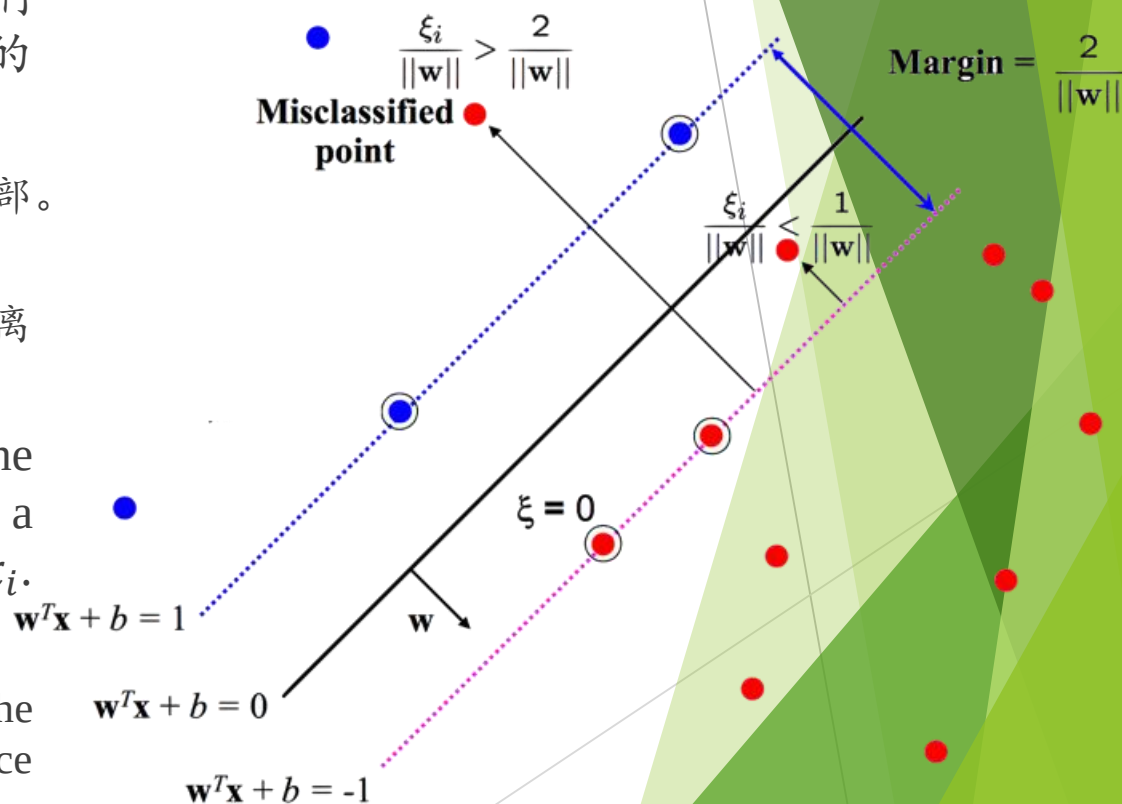
软间隔

Soft Margin

- ▶ 回顾支持向量到决策边界的距离，其为 $1/\|w\|$ ，我们需要基于此，选择合适的非负实数 ξ_i ，对样本点 x_i 的误差进行惩罚，具体分下面两种情况：
 - ▶ 间隔违反：样本位于边界正确的一侧，但落在间隔内部。则它们到边界的距离为 $\xi/\|w\|$ ，其中 $0 < \xi < 1$ ；
 - ▶ 误分类：样本位于边界错误的一侧，它们到边界的距离为 $\xi/\|w\|$ ，其中 $\xi > 1$ 。

Recall that the support vectors have distance $1/\|w\|$ to the decision boundary. Based on this, we want to specify a nonnegative value ξ_i to penalize the error on the point x_i . There are two cases:

- ▶ Margin violation: Points that are on the correct side of the boundary but are inside the margin. They have distance $\xi/\|w\|$, where $0 < \xi < 1$.
- ▶ Misclassification: Points that are on the wrong side of the boundary. They have distance $\xi/\|w\|$, where $\xi > 1$.



软间隔

Soft Margin

- 我们将误差项 ξ_i 引入到优化问题中，具体形式如下：

We incorporate error terms ξ_i 's into our optimization problem by:

$$\begin{cases} \min_{\xi_i \in \mathbb{R}^+, w, b} \|w\|^2 + \lambda \sum_{i=1}^N \xi_i \\ \text{s.t. } y_i(w^T x_i + b) \geq 1 - \xi_i, i = 1, 2, \dots, N \end{cases}$$

该问题的解称为软间隔支持向量分类。

The solution to this problem is called soft margin support vector classification.

- 一般而言，在式中选择不同的 λ 值，将会得到不同的分类器： λ 较小 $\rightarrow$ 对误差的惩罚较轻 $\rightarrow$ 分类器会得到较宽的间隔； λ 较大 $\rightarrow$ 对误差的惩罚较重 $\rightarrow$ 分类器会采用较窄的间隔以提升分类效果。

In general, choosing different values for λ will give us different classifiers: small λ penalizes errors less and hence the classifier will have a large margin; large λ penalizes errors more and hence the classifier will accept narrow margins to improve classification.

核方法

Kernel Trick

- ▶ 给定一个训练集： $\{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ ，在只有一个实值预测变量的情况下，我们可以将拟合一个二次多项式模型的过程，即 $w_0 + w_1x + w_2x^2$ ，看作是在数据上寻找一条最佳拟合数据的二次曲线。但在实际操作中，我们首先对训练集进行特征维度的扩展 $x_i \mapsto (x_i^0, x_i^1, x_i^2)$ ，然后在扩展后的数据 $\{(x_1^0, x_1^1, x_1^2), \dots, (x_N^0, x_N^1, x_N^2)\}$ 上训练一个线性模型。

Given a training set $\{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ with a single real-valued predictor, we can view fitting a 2nd degree polynomial model, aka., $w_0 + w_1x + w_2x^2$, on the data as the process of finding the best quadratic curve that fits the data. But in practice, we first expand the feature dimension of the training set $x_i \mapsto (x_i^0, x_i^1, x_i^2)$ and train a linear model on the expanded data $\{(x_1^0, x_1^1, x_1^2), \dots, (x_N^0, x_N^1, x_N^2)\}$.

- ▶ 我们可以观察到：训练一个多项式模型，本质上就是在经过变换后的预测变量数据上训练一个线性模型。

We can observe that training a polynomial model is just training a linear model on data with transformed predictors.

核方法

Kernel Trick

- 在此例中，为了拟合一个二次多项式模型，需要进行如下映射： $\phi: \mathbb{R} \rightarrow \mathbb{R}^3$ 其中， $x \mapsto (x^0, x^1, x^2)$ 。 $\mathbb{R}$ 称为输入空间， $\mathbb{R}^3$ 称为特征空间。虽然响应变量在输入空间中可能不具有线性相关关系，但在特征空间中，它可能呈现出线性关系。

In our example, transforming the data to fit a 2nd degree polynomial model requires a map: $\phi: \mathbb{R} \rightarrow \mathbb{R}^3$, where $x \mapsto (x^0, x^1, x^2)$, where $\mathbb{R}$ called the input space, $\mathbb{R}^3$ is called the feature space. While the response may not have a linear correlation in the input space, it may have one in the feature space.

- 同样的思想也适用于分类问题：虽然响应变量在输入空间中可能不是线性可分的，但经过某种巧妙的变换之后，在特征空间中它可能变得线性可分。

The same insight applies to classification: while the response may not be linear separable in the input space, it may be in a feature space after a fancy transformation.

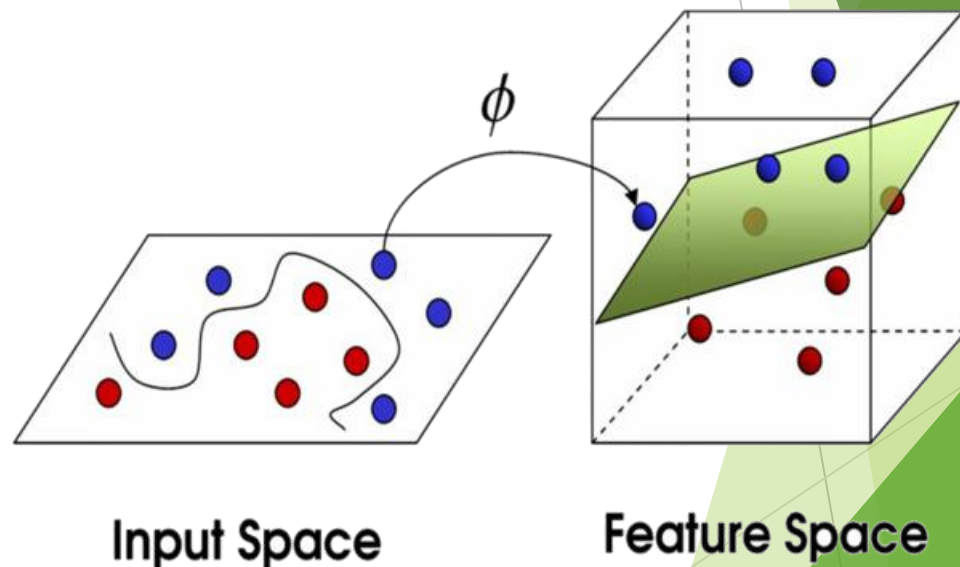
核方法

Kernel Trick

- 因此，与其修改SVC的定义以适应非线性的决策边界，不如将数据映射到一个特征空间，在该空间中类别是线性可分的（或近似线性可分的）。即对训练数据应用变换 $\phi: \mathbb{R}^n \rightarrow \mathbb{R}^m$
 $x \mapsto \phi(x)$ ，其中 $n < m$ ，然后在变换后的数据 $\{(\phi(x_1), y_1), \dots, (\phi(x_N), y_N)\}$ 上训练一个 SVC。

So, instead of tweaking the definition of SVC to accommodate non-linear decision boundaries, we map the data into a feature space in which the classes are linearly separable (or nearly separable):

Apply transform $\phi: \mathbb{R}^n \rightarrow \mathbb{R}^m$ where $n < m$. Train an SVC on the transformed data $\{(\phi(x_1), y_1), \dots, (\phi(x_N), y_N)\}$.



核方法

Kernel Trick

- 由于特征空间可能具有较高的维度，显式地计算 $\phi(x)$ 往往代价非常高昂。我们真正关心的只是特征空间中的内积，而内积是两个向量相似性的度量，因此，我们选择一个核函数 K ，计算 $K(x_i, x_j)$ ，作为 $\phi(x_i)^T \phi(x_j)$ 的内积，而不显式地计算映射 $\phi(x_i)$ 与 $\phi(x_j)$ 。这种在特征空间中训练SVC、却不需要显式处理映射 ϕ 的方法，称为核技巧。

Since the feature space is potentially extremely high dimensional, computing $\phi(x)$ explicitly can be costly. We are only interested in computing inner products, which is a measure of the similarity of the two vectors. So, we choose a kernel K , Computing $K(x_i, x_j)$, treat it as the inner product of mappings $\phi(x_i)$ and $\phi(x_j)$ without explicit computing. This way of training a SVC in feature space without explicitly working with the mapping ϕ is called the kernel trick.

名称 Name	形式 Form	超参 Hyper.
多项式核函数 kernel='poly'	$K(x_i, x_j) = (x_i^T x_j + 1)^d$	d
径向基函数核 kernel='rbf'	$K(x_i, x_j) = \exp \left\{ -\frac{\ x_i - x_j\ ^2}{2\sigma^2} \right\}$	σ
Sigmoid核函数 kernel='sigmoid'	$K(x_i, x_j) = \tanh(\kappa x_i^T x_j + \theta)$	κ, θ